

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Санкт-Петербургская школа физико-математических
и компьютерных наук НИУ ВШЭ — Санкт-Петербург
Образовательная программа «Прикладная математика и информатика»

УДК 004.421

СОГЛАСОВАНО

Научный руководитель,
Старший преподаватель

_____ С. В. Копелиович
« ____ » _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Прикладная математика
и информатика»,
Доцент, канд. техн. наук

_____ А. И. Храбров
« ____ » _____ 2026 г.

Выпускная квалификационная работа
(академическая)

на тему: Улучшенная нижняя оценка для константы Чватала–Санкоффа методом
динамического программирования с окнами

по направлению подготовки 01.03.02 «Прикладная математика и информатика»

ВЫПОЛНИЛ

студент группы БПМ222С
образовательной программы
01.03.02 «Прикладная
математика и информатика»

_____ Д. Е. Юкачев
« ____ » _____ 2026 г.

Аннотация

Работа посвящена получению строгой нижней оценки константы Чватала–Санкоффа γ_σ — предела отношения математического ожидания длины наибольшей общей подпоследовательности (LCS) двух случайных равномерных строк длины n над алфавитом размера σ к n при $n \rightarrow \infty$. Значение γ_σ известно только в виде интервала: для $\sigma = 2$ доказано $\gamma_2 \in [0,7927; 0,8263]$ (Heineman et al., 2024; Lueker, 2009), при этом численные оценки показывают $\gamma_2 \approx 0,811$.

Предложен новый метод вычисления нижних оценок — оконное динамическое программирование с ограниченным обзором N и максимальным сдвигом MAX_SH. Идея доказательства: ДП вычисляет оптимальное среднее число совпадений стратегии с ограниченным обзором за K шагов; по индукции и леммой Фекете о супераддитивных последовательностях нормированный предел $f_K(0, 0, 0)/K$ существует и не превосходит γ_σ , поскольку всякая такая стратегия реализуема и на реальных случайных строках. Сложность алгоритма по времени $O(K \cdot \text{MAX_SH} \cdot \sigma^{2N+2})$, по памяти $O(\text{MAX_SH} \cdot \sigma^{2N})$. Метод универсально применим для произвольного $\sigma \geq 2$. Реализация на C++ с использованием AVX2 и одинарной точности позволила достичь $N = 12$, MAX_SH = 30, $K = 40\,000$ при ≈ 2 ГБ памяти и ≈ 5 часах счёта на одном ядре.

Получены новые рекордные нижние оценки: $\gamma_2 \geq 0,7964$ (прирост +0,48% к Heineman 2024), а также рекорды для всех $\sigma \in \{3, \dots, 10\}$ с приростом от +0,17% до +5,42% по сравнению с лучшими известными результатами. Все оценки независимо верифицированы методом Monte-Carlo на строках длины до 20 000 (по 500 испытаний на σ).

Данная работа состоит из 35 страниц, 5 глав, 2 рисунков, 9 листингов, 8 таблиц, 4 приложений. Использовано 13 источников.

Ключевые слова: константа Чватала–Санкоффа; наибольшая общая подпоследовательность; динамическое программирование; нижние оценки; случайные строки; задача Улама; Monte-Carlo.

Abstract

This work is devoted to establishing a rigorous lower bound for the Chvátal–Sankoff constant γ_σ — the limit of the ratio of the expected length of the longest common subsequence (LCS) of two i.i.d. uniformly random strings of length n over an alphabet of size σ to n as $n \rightarrow \infty$. The value of γ_σ is known only as an interval: for $\sigma = 2$ it is proven that $\gamma_2 \in [0.7927; 0.8263]$ (Heineman et al., 2024; Lueker, 2009), while numerical estimates suggest $\gamma_2 \approx 0.811$.

A new method for computing lower bounds is proposed — windowed dynamic programming with bounded look-ahead N and maximal shift `MAX_SH`. Proof idea: the DP computes the optimal mean number of matches achievable by a bounded-look-ahead strategy over K steps; by induction and Fekete’s superadditivity lemma, the normalized limit $f_K(0,0,0)/K$ exists and is bounded above by γ_σ , since every such strategy is realizable on actual random strings. Algorithm complexity: $O(K \cdot \text{MAX_SH} \cdot \sigma^{2N+2})$ time, $O(\text{MAX_SH} \cdot \sigma^{2N})$ memory. The method is universally applicable for any $\sigma \geq 2$. A C++ implementation using AVX2 and single-precision arithmetic allows $N = 12$, `MAX_SH` = 30, $K = 40\,000$ with ≈ 2 GB memory and ≈ 5 hours on a single core.

New record lower bounds are obtained: $\gamma_2 \geq 0.7964$ (improvement of +0.48% over Heineman 2024), as well as records for all $\sigma \in \{3, \dots, 10\}$ with improvements from +0.17% to +5.42% compared to the best known results. All bounds are independently verified by Monte-Carlo simulation on strings of length up to 20 000 (with 500 trials per σ).

The paper contains 35 pages, 5 chapters, 2 figures, 9 listings, 8 tables, 4 appendices. 13 sources are used.

Keywords: Chvátal–Sankoff constant; longest common subsequence; dynamic programming; lower bounds; random strings; Ulam problem; Monte-Carlo.

Содержание

Аннотация	2
Abstract	3
Используемые определения и термины	6
Введение	8
1 Предметная область и обзор литературы	11
1.1 Задача о наибольшей общей подпоследовательности	11
1.2 Случайная модель	11
1.3 Константа Чватала–Санкоффа	11
1.4 Связь с задачей Улама	12
1.5 Хронология нижних оценок γ_2	12
1.6 Верхние оценки	12
1.7 Эмпирические оценки	13
1.8 Программные инструменты и предшествующие реализации	13
Выводы по разделу 1	13
2 Метод оконного динамического программирования	14
2.1 Идея стратегии с ограниченным обзором	14
2.2 Кодирование состояния	14
2.3 ДП-уравнения	15
2.4 Граничные условия по сдвигу	16
2.5 Универсализация на алфавит произвольного размера	16
2.6 Длина симуляции	16
Выводы по разделу 2	16
3 Корректность и вычислительная сложность	18
3.1 Лемма о нижней оценке через стратегию	18
3.2 Связь со средним	18
3.3 Теорема о корректности ДП	19
3.4 Теорема о сходимости	20
3.5 Анализ вычислительной сложности	20
3.6 Источники численной погрешности	21
Выводы по разделу 3	21
4 Реализация и история оптимизаций	22
4.1 Базовая реализация (версии x0–x2)	22
4.2 Разделение размера окна и сдвига (x3)	22

4.3	Развёртка циклов и большие параметры (x4)	22
4.4	Переход на одинарную точность и AVX2 (x5)	22
4.5	Финальная версия (x6, бинарный алфавит)	23
4.6	Таблица эволюции версий	24
4.7	Универсальная версия (windowed_dp_universal.cpp)	24
4.8	Проверка эквивалентности версий	25
4.9	Программная инфраструктура	25
	Выводы по разделу 4	26
5	Численные эксперименты и верификация	27
5.1	Постановка экспериментов	27
5.2	Главный результат: $\gamma_2 \geq 0,7964$	27
5.3	Результаты для $\sigma = 3, \dots, 10$	28
5.4	Зависимость от параметров метода	28
5.5	Верификация Monte-Carlo	29
5.6	Сравнение с историей задачи	30
	Выводы по разделу 5	30
	Заключение	32
	Список использованных источников	35
	Приложение А	36
	Приложение Б	38
	Приложение В	40
	Приложение Г	45

Используемые определения и термины

$f_K(\text{sh}, a, b)$ – максимальное ожидаемое число совпадений за K шагов из состояния (sh, a, b) при оптимальной стратегии; центральная ДП-функция работы, определяется в §2.3.

AVX2 (Advanced Vector Extensions 2) – расширение набора инструкций x86, поддерживающее 256-битные векторные операции с целыми числами и числами с плавающей точкой.

Алфавит Σ_σ – конечное непустое множество символов фиксированного размера $\sigma \geq 2$; в работе используется $\Sigma_\sigma = \{0, 1, \dots, \sigma - 1\}$.

Валидная маска – целое число $x \in [0, 2^{N \cdot \text{BITS}})$, в котором каждый из N блоков по $\text{BITS} = \lceil \log_2 \sigma \rceil$ бит кодирует символ из Σ_σ (т.е. лежит в $[0, \sigma)$); множество валидных масок имеет размер σ^N .

Динамическое программирование (ДП, Dynamic Programming, DP) – метод решения задач путём разложения их на подзадачи и запоминания решений подзадач для повторного использования.

Константа Чватала–Санкоффа γ_σ – предел отношения $\mathbb{E}[L_n]/n$ при $n \rightarrow \infty$, где L_n – длина наибольшей общей подпоследовательности двух независимых равномерно случайных строк длины n над алфавитом размера σ .

Лемма Фекете – утверждение о том, что для любой супераддитивной последовательности $\{g_n\}$ (т.е. $g_{n+m} \geq g_n + g_m$) существует предел $\lim_{n \rightarrow \infty} g_n/n = \sup_n g_n/n \in (-\infty, +\infty]$.

Маска (состояния окна) – целочисленное упакованное представление N -символьного окна одной из строк: $a = \sum_{p=0}^{N-1} a_p \cdot 2^{p \cdot \text{BITS}}$, где $a_p \in \Sigma_\sigma$ – p -й символ окна (нумерация от младшей позиции).

Метод Monte-Carlo – метод приближённого вычисления математических величин с помощью многократной случайной выборки и статистического усреднения; в работе используется для эмпирической оценки γ_σ через классический ДП-LCS.

Наибольшая общая подпоследовательность (LCS, Longest Common Subsequence) – максимальная по длине последовательность символов, которая является подпоследовательностью каждой из двух (или более) данных строк; длина LCS пары (A, B) обозначается $\text{lcs}(A, B)$.

Окно (look-ahead window) – в данной работе – упорядоченный набор из N ближайших ещё не использованных символов каждой из двух строк, видимый стратегии в текущий момент; в реализации кодируется маской (см. §2.2).

Подпоследовательность – строка, полученная из данной вычёркиванием некоторого (возможно, пустого) подмножества символов с сохранением порядка оставшихся.

Сдвиг (shift, sh, SH) – разность $j - i$ позиций, на которых в данный момент стоят указатели по второй и первой строкам; в работе ограничен значениями из отрезка $[0, \text{MAX_SH}]$.

Состояние ДП – в данной работе – тройка (sh, a, b) , где $\text{sh} \in [0, \text{MAX_SH}]$ – текущий сдвиг, a и b – маски следующих N символов первой и второй строк соответственно; множество допустимых состояний обозначается $\mathcal{S}$.

Стратегия с ограниченным обзором (стратегия π) – детерминированный или случайный алгоритм построения общей подпоследовательности двух строк, выбирающий на каждом шаге одно из трёх действий – сдвиг по первой строке (L), сдвиг по второй строке (R) или засчитывание совпадения младших символов окон – как функцию текущего состояния (sh, a, b) (см. §2.1).

Супераддитивность – свойство последовательности $\{g_n\}$, состоящее в выполнении неравенства $g_{n+m} \geq g_n + g_m$ для всех $n, m \geq 0$; обеспечивает применимость леммы Фекете.

Введение

Предметная область

Задача о наибольшей общей подпоследовательности (Longest Common Subsequence, LCS) — одна из фундаментальных задач алгоритмики и комбинаторики на словах. Классический алгоритм Вагнера–Фишера [1] вычисляет длину LCS двух строк длины n за время $O(n^2)$. Помимо непосредственных приложений (системы контроля версий, утилита diff, выравнивание биологических последовательностей), LCS играет роль модельной задачи в теории случайных процессов на словах.

Пусть $A = a_1 a_2 \dots a_n$ и $B = b_1 b_2 \dots b_n$ — независимые случайные строки, каждая буква которых выбирается равномерно и независимо из алфавита размера σ . Пусть L_n — длина LCS этих строк. В 1975 году Чватал и Санкофф [2] доказали, что существует предел

$$\gamma_\sigma := \lim_{n \rightarrow \infty} \frac{\mathbb{E}[L_n]}{n}.$$

Эта величина называется константой Чватала–Санкоффа. Точное значение γ_σ не известно ни при каком σ и является классической открытой проблемой [3; 4].

Объект и предмет исследования

Объект исследования — случайная величина L_n длины LCS двух независимых равномерно случайных строк длины n над алфавитом размера σ .

Предмет исследования — предел $\gamma_\sigma = \lim_{n \rightarrow \infty} \mathbb{E}[L_n]/n$ и строгие нижние оценки этой величины, получаемые конечнокомбинаторными методами.

Актуальность

Для бинарного алфавита $\sigma = 2$ известные строгие оценки таковы:

$$0,7927 \leq \gamma_2 \leq 0,8263,$$

где нижняя оценка получена Heineman et al. в [4] (2024), верхняя — Lueker’ом в [3] (2009). Эмпирические оценки методом Monte-Carlo показывают $\gamma_2 \approx 0,811$, что указывает на существенный разрыв нижней оценки с истинным значением. С 2003 по 2024 год нижняя оценка γ_2 улучшалась медленно: суммарный прирост за два десятилетия составил всего $\approx 0,005$. Каждое новое улучшение, превышающее 10^{-3} , представляет самостоятельный интерес.

Аналогичный разрыв имеет место и для $\sigma \geq 3$. Существующие подходы (конечные автоматы, расширенные пространства состояний) упираются в экспоненциальный рост пространства состояний, что мотивирует поиск качественно новых вычислительных схем.

Принципиальное преимущество предлагаемого в работе подхода — разделение двух параметров: размера окна обзора N и максимального сдвига MAX_SH . Это позволяет наращивать выразительность стратегии (рост N) и допустимый «зазор» между указателями (рост MAX_SH) независимо, удерживая память в управляемых пределах $O(\text{MAX_SH} \cdot \sigma^{2N})$, тогда как в схемах с единым параметром пространство состояний растёт как $\sigma^{O(N^2)}$ или быстрее.

Цель и задачи

Цель работы — получить строгую нижнюю оценку γ_σ , превосходящую известные результаты, для $\sigma \in \{2, 3, \dots, 10\}$.

Задачи:

- 1) Сформулировать оконную стратегию построения общей подпоследовательности двух случайных строк и доказать, что её ожидаемая нормированная длина является нижней оценкой γ_σ .
- 2) Разработать метод динамического программирования, позволяющий точно вычислить оптимальную ожидаемую длину в классе оконных стратегий.
- 3) Доказать корректность ДП-уравнений и оценить асимптотическую сложность алгоритма по памяти и времени.
- 4) Реализовать алгоритм на C++ с современными оптимизациями (одинарная точность, AVX2, развёртка циклов).
- 5) Провести масштабные численные эксперименты для $\sigma \in \{2, \dots, 10\}$.
- 6) Независимо верифицировать результаты методом Monte-Carlo.
- 7) Сравнить полученные оценки с лучшими опубликованными результатами.

Методы исследования

В работе используются: формальное доказательство корректности рекуррентных уравнений ДП индукцией по числу шагов; лемма Фекете [5] о супераддитивных последовательностях для обоснования существования предела f_K/K ; численные эксперименты на языке C++ с использованием векторных расширений AVX2; независимая проверка методом Monte-Carlo по классическому ДП-алгоритму LCS на 500 парах случайных строк длины до 20 000 для каждого σ .

Основные результаты

- Получена рекордная нижняя оценка $\gamma_2 \geq 0,7964$ (предыдущий рекорд 0,79266 [4]).

- Получены рекордные нижние оценки γ_σ для всех $\sigma \in \{3, \dots, 10\}$ с приростом от $+0,17\%$ до $+5,42\%$ относительно лучших известных результатов.
- Все оценки независимо подтверждены методом Monte-Carlo.

Структура работы

Работа состоит из пяти разделов. В разделе 1 даётся обзор предметной области и сводка известных результатов. В разделе 2 формулируется метод оконного ДП. В разделе 3 доказываемся корректность метода и анализируется его сложность. Раздел 4 описывает реализацию и историю оптимизаций. Раздел 5 содержит результаты численных экспериментов и Monte-Carlo верификацию.

1. Предметная область и обзор литературы

1.1. Задача о наибольшей общей подпоследовательности

Определение 1.1 (подпоследовательность). Подпоследовательностью строки $A = a_1 a_2 \dots a_n$ называется строка вида $a_{i_1} a_{i_2} \dots a_{i_k}$, где $1 \leq i_1 < i_2 < \dots < i_k \leq n$.

Определение 1.2 (LCS). Для двух строк A и B их наибольшая общая подпоследовательность (LCS) — это подпоследовательность, являющаяся одновременно подпоследовательностью A и B и имеющая максимально возможную длину. Длина LCS обозначается $\text{lcs}(A, B)$. Заметим, что сама LCS может быть не единственной, однако её длина $\text{lcs}(A, B)$ определена однозначно.

Классический ДП-алгоритм Вагнера–Фишера [1] вычисляет $\text{lcs}(A, B)$ за время $O(|A| \cdot |B|)$ и память $O(\min(|A|, |B|))$ по рекуррентному соотношению

$$f[i][j] = \begin{cases} 0, & i = 0 \text{ или } j = 0, \\ f[i-1][j-1] + 1, & a_i = b_j, \\ \max(f[i-1][j], f[i][j-1]), & a_i \neq b_j. \end{cases}$$

Корректность рекуррентного соотношения следует из оптимальной подструктуры: при $a_i = b_j$ оптимальное решение пары $(A[1..i], B[1..j])$ продолжает LCS пары $(A[1..i-1], B[1..j-1])$ символом $a_i = b_j$; при $a_i \neq b_j$ хотя бы один из последних символов не входит в LCS, что даёт максимум по двум подзадачам. Оценка памяти $O(\min(|A|, |B|))$ достигается за счёт хранения только двух соседних строк ДП-таблицы.

1.2. Случайная модель

Пусть $\sigma \geq 2$ — размер алфавита, обозначим алфавит $\Sigma_\sigma = \{0, 1, \dots, \sigma - 1\}$. Случайной моделью LCS называется выбор двух независимых строк $A, B \in \Sigma_\sigma^n$, в которых каждый символ независимо и равновероятно выбирается из Σ_σ . Длина LCS становится при этом случайной величиной, обозначаемой $L_n = L_n^{(\sigma)}$.

1.3. Константа Чватала–Санкоффа

Теорема 1.1 (Чватал, Санкофф, 1975 [2]). Существует предел

$$\gamma_\sigma := \lim_{n \rightarrow \infty} \frac{\mathbb{E}[L_n^{(\sigma)}]}{n} \in [0, 1].$$

Существование предела следует из супераддитивности математического ожидания: $\mathbb{E}[L_{n+m}] \geq \mathbb{E}[L_n] + \mathbb{E}[L_m]$. Действительно, если в первых n символах найдена общая подпоследовательность длины ℓ_1 , а во вторых m независимо — длины ℓ_2 , их конкатенация даёт общую подпоследовательность пары полных строк длины $\ell_1 + \ell_2$, что и

означает супераддитивность по математическому ожиданию. Применение леммы Фекете о супераддитивных последовательностях [5] гарантирует существование предела $\lim_{n \rightarrow \infty} \mathbb{E}[L_n]/n$. Точное значение γ_σ не известно ни при каком σ .

Стиил [6] установил, что L_n сильно концентрируется около своего среднего: $\mathbb{D}[L_n] = O(n)$, откуда $L_n/n \rightarrow \gamma_\sigma$ почти наверное. Иными словами, длина LCS асимптотически линейна с коэффициентом γ_σ .

1.4. Связь с задачей Улама

Задача Улама о длине наибольшей возрастающей подпоследовательности случайной перестановки длины n имеет известную асимптотику $2\sqrt{n}$ [7–9]. LCS двух случайных строк — естественное обобщение задачи Улама в дискретный алфавит. Асимптотика γ_σ при $\sigma \rightarrow \infty$ имеет вид [10]

$$\gamma_\sigma \sim \frac{2}{\sqrt{\sigma}}, \quad \sigma \rightarrow \infty.$$

Интуитивно, при $\sigma \rightarrow \infty$ большинство символов в двух случайных строках различны, и задача поиска LCS приближается к задаче нахождения наибольшей возрастающей подпоследовательности после идентификации совпадающих пар индексов, что и объясняет асимптотику $2/\sqrt{\sigma}$.

1.5. Хронология нижних оценок γ_2

В таблице 1 приведена эволюция известных нижних оценок γ_2 за полвека.

Таблица 1 — Хронология строгих нижних оценок γ_2

Год	Авторы	$\gamma_2 \geq$	Метод
1975	Chvátal, Sankoff [2]	0,7615	Жадная стратегия
1995	Dančák, Paterson [11]	0,7739	Конечные автоматы
2003	Lueker [12]	0,7881	Двойственная LP-схема
2009	Lueker [3]	0,788	Обзор и улучшение
2024	Heineman et al. [4]	0,79266	Расширенные пространства состояний
2026	Данная работа	0,7964	Оконное ДП

За последние два десятилетия среднегодовой прирост нижней оценки составляет менее 0,00025, что свидетельствует о сложности задачи и о необходимости принципиально новых вычислительных схем.

1.6. Верхние оценки

Лучшая известная верхняя оценка для бинарного алфавита — $\gamma_2 \leq 0,826280$ (Lueker, 2009 [3]). Метод Lueker’a основан на двойственной формулировке Данчика–Патерсона, усиленной компьютерным перебором. Разрыв между лучшей нижней и верхней оценкой γ_2 составляет $\approx 0,034$.

1.7. Эмпирические оценки

Численные эксперименты методом Monte-Carlo (классический ДП-LCS на парах независимых строк длины $n = 10\,000\text{--}20\,000$, усреднение по 500 реализациям, стандартная ошибка среднего порядка 10^{-3}) дают следующие приближённые значения константы:

- $\gamma_2 \approx 0,811$,
- $\gamma_3 \approx 0,716$,
- $\gamma_4 \approx 0,653$,
- $\gamma_5 \approx 0,606$,

с дальнейшим монотонным убыванием в соответствии с асимптотикой $\gamma_\sigma \sim 2/\sqrt{\sigma}$. Эти значения служат верхним ориентиром для качества строгих нижних оценок: ни одна нижняя оценка не может превосходить эмпирического среднего, и чем меньше зазор, тем «качественнее» метод. Полный набор оценок и параметров Monte-Carlo приведён в §5.3.

1.8. Программные инструменты и предшествующие реализации

Авторы работы [4] опубликовали свой код и данные на GitHub: <https://github.com/Statistics-of-Subsequences/PaperMaterials>. Это позволяет сравнивать численные результаты на единых параметрах.

Исходные коды настоящей работы (реализация оконного ДП, универсальная версия для произвольного σ , Monte-Carlo верификация) доступны в открытом репозитории: <https://github.com/Youka4/HSEDiploma>.

Выводы по разделу

- 1) Задача оценки константы Чватала–Санкоффа γ_σ — классическая открытая проблема комбинаторики на словах, известная с 1975 года.
- 2) Лучшие строгие оценки для бинарного алфавита: $0,7927 \leq \gamma_2 \leq 0,8263$; эмпирически $\gamma_2 \approx 0,811$. Аналогичные разрывы имеют место для $\sigma \geq 3$.
- 3) За 2003–2024 годы суммарный прирост нижней оценки γ_2 составил всего $\approx 0,005$. Существующие методы (конечные автоматы, расширенные пространства состояний) упираются в их экспоненциальный рост.
- 4) Для дальнейшего продвижения требуется новый класс вычислительных схем, дающий улучшение порядка 10^{-3} или более.

2. Метод оконного динамического программирования

2.1. Идея стратегии с ограниченным обзором

Зафиксируем размер алфавита $\sigma \geq 2$, целое $N \geq 1$ (размер окна) и целое $\text{MAX_SH} \geq 0$ (максимальный сдвиг). Пусть даны две независимые равномерно случайные строки A, B над алфавитом Σ_σ достаточно большой длины.

Стратегия с ограниченным обзором — это алгоритм, по шагам строящий общую подпоследовательность строк A и B . На каждом шаге алгоритм имеет два указателя i (по строке A) и j (по строке B), при этом доступны:

- окно следующих N символов первой строки: $a_i, a_{i+1}, \dots, a_{i+N-1}$;
- окно следующих N символов второй строки: $b_j, b_{j+1}, \dots, b_{j+N-1}$;
- текущий сдвиг $\text{sh} \in [0, \text{MAX_SH}]$ (содержательно — это нормированная разность $j - i$, ограниченная допустимым окном; см. §2.2 о нормировке).

Действия алгоритма на каждом шаге:

- 1) Если $a_i = b_j$: алгоритм обязан засчитать это совпадение: прибавить $+1$ к длине общей подпоследовательности и сдвинуть оба указателя на 1, то есть $i \rightarrow i + 1$, $j \rightarrow j + 1$.
- 2) Иначе $a_i \neq b_j$: алгоритм выбирает одно из двух действий:
 - (L) сдвинуть только i : $i \rightarrow i + 1$, $\text{sh} \rightarrow \text{sh} - 1$;
 - (R) сдвинуть только j : $j \rightarrow j + 1$, $\text{sh} \rightarrow \text{sh} + 1$.

Однако, если sh уже равен крайнему значению, разрешённое действие однозначно определено: например, при $\text{sh} = \text{MAX_SH}$ запрещено действие (R).

Длина общей подпоследовательности, построенной такой стратегией за T шагов, обозначается S_T^π , где π — стратегия. По построению $S_T^\pi \leq \text{lcs}(A_{1..T_A}, B_{1..T_B})$, где T_A, T_B — финальные значения указателей.

2.2. Кодирование состояния

Каждый символ алфавита Σ_σ представляется $\text{BITS} = \lceil \log_2 \sigma \rceil$ битами. Окно из N символов первой строки кодируется числом $a \in \{0, 1, \dots, \sigma^N - 1\}$, в общем случае помещающимся в $\text{MASK_BITS} = \text{BITS} \cdot N$ бит. Аналогично кодируется окно b второй строки.

Нормировка сдвига. В реализации сдвиг sh кодируется неотрицательным целым числом в диапазоне $[0, \text{MAX_SH}]$, представляющим текущую разность $j - i$ относительно базового опорного значения; параметр MAX_SH задаёт допустимый размер этого

диапазона. При таком соглашении границы $sh = 0$ и $sh = \text{MAX_SH}$ соответствуют выходу за допустимую полосу: при $sh = 0$ запрещено действие (L), уменьшающее сдвиг, при $sh = \text{MAX_SH}$ — действие (R), увеличивающее сдвиг (см. §2.4). В коде это даёт массив ДП размера $\text{MAX_SH} + 1$ слоёв.

Состояние ДП — тройка $s = (sh, a, b)$. Общее число состояний составляет $(\text{MAX_SH} + 1) \cdot \sigma^{2N}$.

2.3. ДП-уравнения

Обозначим через $f_K(sh, a, b)$ ожидаемое значение длины общей подпоследовательности, построенной оптимальной стратегией за K шагов, начиная с состояния (sh, a, b) . Базовый случай:

$$f_0(sh, a, b) = 0 \quad \text{для всех } (sh, a, b).$$

Для шага $K \geq 1$ рекуррентное соотношение имеет вид:

$$f_K(sh, a, b) = \mathbb{E}_{a_0, b_0 \sim U(\Sigma_\sigma)} [T_K(sh, a, b, a_0, b_0)],$$

где математическое ожидание берётся по новой паре символов (a_0, b_0) , поступающих в концы окон, а функция T_K описывает оптимальный выбор действия:

$$T_K(sh, a, b, a_0, b_0) = \begin{cases} 1 + f_{K-1}(sh, a', b'), & \text{если } a \bmod \sigma = b \bmod \sigma, \\ \max(f_{K-1}(sh - 1, a', b), f_{K-1}(sh + 1, a, b')), & \text{иначе,} \end{cases}$$

где a' получается из a удалением младшего символа и добавлением нового символа a_0 в старший разряд:

$$a' = (a \gg \text{BITS}) \mid (a_0 \ll (\text{MASK_BITS} - \text{BITS})),$$

аналогично определяется b' .

В ветви совпадения (символы a_i и b_j совпадают, т.е. $a \bmod \sigma = b \bmod \sigma$) оба окна сдвигаются на одну позицию; в ветви несовпадения алгоритм выбирает, какое из окон сдвинуть.

Порядок принятия решения. Подчёркиваем, что выбор действия в ветви несовпадения (то есть взятие $\max$) производится до того, как становятся известны новые символы a_0, b_0 : алгоритм видит лишь текущие окна (a, b) и сдвиг sh . Соответственно в формуле для f_K математическое ожидание $\mathbb{E}_{a_0, b_0}[\cdot]$ берётся снаружи от выражения T_K , содержащего $\max$. Так как $\max(f_{K-1}(sh - 1, a', b), f_{K-1}(sh + 1, a, b'))$ не зависит от (a_0, b_0) в той ветви, где требуется продолжение использует только одно из обновлённых окон (см. уравнения выше), но в общем случае a' и b' являются функциями от a_0 и b_0 , и порядок $\mathbb{E}[\max(\cdot)]$ (а не $\max(\mathbb{E}[\cdot])$) существует.

2.4. Граничные условия по сдвигу

При $sh = 0$ запрещено действие (L), уменьшающее сдвиг ниже нуля; при $sh = \text{MAX_SH}$ запрещено действие (R), увеличивающее сдвиг сверх лимита. В этих случаях в несовпадающей ветви максимум вырождается:

- при $sh = 0, a \bmod \sigma \neq b \bmod \sigma: T_K = f_{K-1}(1, a, b')$ (доступно только действие (R));
- при $sh = \text{MAX_SH}, a \bmod \sigma \neq b \bmod \sigma: T_K = f_{K-1}(\text{MAX_SH} - 1, a', b)$ (доступно только действие (L)).

В реализации эти случаи учитываются заведением вспомогательных «зеркальных» массивов m_0 (для $sh = 0$) и m_1 (для $sh = \text{MAX_SH}$), хранящих значения предыдущего слоя в нужных «копиях».

2.5. Универсализация на алфавит произвольного размера

Для σ , не являющегося степенью двойки, не все значения битовой маски размера MASK_BITS соответствуют допустимым окнам (например, при $\sigma = 3$ маска 0b11 соответствует символу $3 \notin \Sigma_3$). Чтобы избежать обхода недопустимых состояний, на этапе инициализации строится список валидных масок $\mathcal{V} \subseteq \{0, 1, \dots, 2^{\text{MASK_BITS}} - 1\}$ размера σ^N .

При $\sigma = 9, N = 4: \text{MASK_BITS} = 16$, общее число масок $2^{16} = 65\,536$, из которых валидных — $9^4 = 6\,561$.

Малый пример: $\sigma = 3, N = 5$. Здесь $\text{BITS} = 2, \text{MASK_BITS} = 10$, общее число масок $2^{10} = 1024$. Валидны те маски, в которых каждый из пяти двухбитовых блоков лежит в $\{00, 01, 10\}$; их $3^5 = 243$, то есть около 24 % от общего числа. Например, маска 0b 01 10 00 01 10 (равная 390) валидна и кодирует окно $(2, 1, 0, 2, 1)$, тогда как маска с блоком 11 = 3 невалидна и в ДП-итерации не появляется.

2.6. Длина симуляции

При запуске стратегии симулируется некоторое число шагов K , которое играет роль «длины строки». Очевидно, $f_K(0, 0, 0)/K$ — ожидаемая нормированная длина общей подпоследовательности оптимальной оконной стратегии при симуляции K шагов. В разделе 3 будет показано, что предел этой величины при $K \rightarrow \infty$ существует и не превосходит γ_σ , что и обеспечивает строгость нижней оценки.

Выводы по разделу

- 1) Сформулирована стратегия с ограниченным обзором: алгоритм видит окна из N ближайших символов двух строк и текущий сдвиг, ограниченный значением MAX_SH .

- 2) Состояние ДП $s = (\text{sh}, a, b)$ имеет размер $(\text{MAX_SH} + 1) \cdot \sigma^{2N}$.
- 3) Рекуррентное соотношение явно выписано как математическое ожидание по двум независимым новым символам, поступающим в концы окон; граничные условия по сдвигу даны в явной форме.
- 4) Универсализация на произвольный σ достигается за счёт предвычисления списка валидных масок.

3. Корректность и вычислительная сложность

В данном разделе доказывается, что величина, вычисляемая методом оконного ДП из раздела 2, является строгой нижней оценкой константы γ_σ , и анализируется вычислительная сложность алгоритма по памяти и времени.

Обозначения. Через γ_σ обозначим истинную константу Чватала–Санкоффа для алфавита Σ_σ (см. определение 1.1, §1.3). Через $\gamma_{\sigma, N, \text{MAX_SH}}$ — предел нормированной ДП-функции при фиксированных параметрах окна N и максимального сдвига MAX_SH (определяется ниже в теореме 3.4). Для произвольной допустимой стратегии π обозначим $\gamma^\pi := \lim_{T \rightarrow \infty} \mathbb{E}[S_T^\pi]/T$, если предел существует.

3.1. Лемма о нижней оценке через стратегию

Лемма 3.1. Пусть π — произвольная допустимая стратегия с ограниченным обзором (определена в §2.1), T — число её шагов, S_T^π — длина построенной общей подпоследовательности, T_A, T_B — финальные значения указателей. Тогда

$$S_T^\pi \leq \text{lcs}(A_{1..T_A}, B_{1..T_B}).$$

Доказательство. По построению стратегия π строит конкретную последовательность совпадений символов $a_{i_1} = b_{j_1}, a_{i_2} = b_{j_2}, \dots, a_{i_k} = b_{j_k}$, где $i_1 < i_2 < \dots < i_k$ и $j_1 < j_2 < \dots < j_k$: действия (L), (R) и засчитывание совпадения строго увеличивают соответствующий указатель и никогда не уменьшают его, поэтому индексы совпадений строго монотонно возрастают по обоим строкам. Эта последовательность задаёт общую подпоследовательность строк $A_{1..T_A}$ и $B_{1..T_B}$ длины $S_T^\pi = k$. Так как lcs — максимум по всем общим подпоследовательностям, требуемое неравенство выполнено. $\square$

3.2. Связь со средним

Применяя лемму 3.1 к случайным строкам A, B и беря математическое ожидание по обоим строкам, получаем:

$$\mathbb{E}_\pi[S_T^\pi] \leq \mathbb{E}[\text{lcs}(A_{1..T_A}, B_{1..T_B})] \leq \mathbb{E}[\text{lcs}(A_{1..T}, B_{1..T})] = \mathbb{E}[L_T].$$

Второе неравенство — монотонность LCS по префиксу: для любых строк X, X', Y, Y' с $X \preceq X', Y \preceq Y'$ (где $\preceq$ обозначает префиксное вложение) любая общая подпоследовательность пары (X, Y) является общей и для пары (X', Y') , откуда $\text{lcs}(X, Y) \leq \text{lcs}(X', Y')$. При $T_A \leq T, T_B \leq T$ это и даёт требуемое. Деление на T и переход к пределу $T \rightarrow \infty$ даёт

$$\limsup_{T \rightarrow \infty} \frac{\mathbb{E}[S_T^\pi]}{T} \leq \gamma_\sigma.$$

Следствие 3.2. Любая стратегия π с ограниченным обзором, дающая $\mathbb{E}[S_T^\pi]/T \rightarrow$

γ^π при $T \rightarrow \infty$, удовлетворяет $\gamma^\pi \leq \gamma_\sigma$.

3.3. Теорема о корректности ДП

Теорема 3.3 (корректность ДП). Пусть $f_K(\text{sh}, a, b)$ определена рекуррентным соотношением из §2.3 с граничными условиями §2.4. Тогда $f_K(\text{sh}, a, b)$ есть в точности максимальное по всем допустимым стратегиям π значение $\mathbb{E}_\pi[S_K^\pi]$, при условии что в начальный момент состояние есть (sh, a, b) , и за K шагов в концы окон поступают независимые равномерные на Σ_σ символы.

Доказательство. Индукция по K .

База ($K = 0$): стратегия не делает ни одного шага, $S_0^\pi = 0$. Значение $f_0 = 0$ совпадает.

Шаг. Предположим, утверждение верно для $K - 1$, докажем для K . Рассмотрим состояние (sh, a, b) . Пусть (a_0, b_0) — новые символы, поступающие в окна на текущем шаге, $a_0, b_0 \sim U(\Sigma_\sigma)$ независимо. Обозначим младшие символы окон $u = a \bmod \sigma$ (это a_i), $v = b \bmod \sigma$ (это b_j). Возможны два случая.

Случай 1: $u = v$. По правилам стратегии (см. §2.1) алгоритм обязан засчитать совпадение пары (u, v) и сдвинуть оба указателя. Новое состояние — (sh, a', b') , где $a' = (a \gg \text{BITS}) \mid (a_0 \ll (\text{MASK_BITS} - \text{BITS}))$, аналогично b' . Текущий вклад — $+1$. По индукционному предположению ожидаемое продолжение есть $f_{K-1}(\text{sh}, a', b')$. Полное значение:

$$T_K = 1 + f_{K-1}(\text{sh}, a', b'),$$

что совпадает с формулой §2.3.

Случай 2: $u \neq v$. Стратегия выбирает между действиями (L) и (R). Действие (L) переводит в состояние $(\text{sh} - 1, a', b)$, действие (R) — в $(\text{sh} + 1, a, b')$. Текущий вклад — 0 . Оптимальный выбор:

$$T_K = \max(f_{K-1}(\text{sh} - 1, a', b), f_{K-1}(\text{sh} + 1, a, b')).$$

При $\text{sh} = 0$ или $\text{sh} = \text{MAX_SH}$ соответствующая ветвь недоступна и максимум вырождается в одно слагаемое, как описано в §2.4.

Так как (a_0, b_0) независимы от прошлого, операция $\mathbb{E}[\cdot]$ коммутрует с детерминированным $\max$ при условии независимости. Полное:

$$f_K(\text{sh}, a, b) = \mathbb{E}_{a_0, b_0}[T_K] = \sum_{a_0, b_0 \in \Sigma_\sigma} \frac{1}{\sigma^2} \cdot T_K,$$

что и доказывает шаг индукции. □

3.4. Теорема о сходимости

Теорема 3.4 (нижняя оценка). Обозначим $s_0 := (0, 0, 0)$ — начальное состояние ДП. Тогда:

- (i) для любого $K \geq 1$ выполнено $f_K(s_0)/K \leq \mathbb{E}[L_K]/K$;
- (ii) в частности,

$$\limsup_{K \rightarrow \infty} \frac{f_K(s_0)}{K} \leq \gamma_\sigma;$$

то есть для любого $\varepsilon > 0$ и достаточно большого K выполнено $f_K(s_0)/K \leq \gamma_\sigma + \varepsilon$.

Доказательство. По теореме 3.3 значение $f_K(s_0)$ равно $\mathbb{E}_\pi[S_K^\pi]$ для оптимальной стратегии $\pi = \pi_K^*$ с горизонтом K , стартовой из состояния s_0 на случайных строках A, B . Эта стратегия допустима в смысле определения §2.1, поэтому к ней применима лемма 3.1 и вывод из §3.2 о связи со средним:

$$\frac{f_K(s_0)}{K} = \frac{\mathbb{E}[S_K^{\pi_K^*}]}{K} \leq \frac{\mathbb{E}[L_K]}{K}.$$

Это даёт (i). Утверждение (ii) получается переходом к пределу $K \rightarrow \infty$ и применением теоремы 1.1: $\mathbb{E}[L_K]/K \rightarrow \gamma_\sigma$. $\square$

Замечание. Существование самого предела $\lim_{K \rightarrow \infty} f_K(s_0)/K$ для обоснования нижней оценки γ_σ не требуется: согласно пункту (i) теоремы 3.4, каждое конкретное значение $f_K(s_0)/K$, полученное при фиксированном K , уже не превосходит $\mathbb{E}[L_K]/K \leq \gamma_\sigma$ (с точностью до известной эмпирической оценки разности $\gamma_\sigma - \mathbb{E}[L_K]/K$, см. §5.4). Поэтому каждое отдельное вычисление $f_K(s_0)/K$ даёт строгую нижнюю оценку константы Чватала–Санкоффа независимо от вопроса о существовании предела.

Следствие 3.5. Для любых $N \geq 1$, $\text{MAX_SH} \geq 0$ и достаточно большого K выполнено

$$\frac{f_K(0, 0, 0)}{K} \leq \gamma_\sigma + \varepsilon$$

с погрешностью $\varepsilon = \varepsilon(K) \rightarrow 0$. На практике сходимость к пределу наблюдается экспоненциально быстро (см. раздел 5).

3.5. Анализ вычислительной сложности

Память. Для слоя ДП фиксированного значения sh требуется массив размера $\sigma^N \times \sigma^N = M^2$, где $M = \sigma^N$. Хранятся слои для всех $\text{sh} \in [0, \text{MAX_SH}]$ — это $\text{MAX_SH} + 1$ слой — и два вспомогательных «зеркальных» слоя m_0, m_1 для корректной обработки граничных случаев $\text{sh} = 0$ и $\text{sh} = \text{MAX_SH}$ без условных переходов (см. §2.4). Итого хранится $\text{MAX_SH} + 1 + 2 = \text{MAX_SH} + 3$ слоёв; в асимптотике это $O(\text{MAX_SH} \cdot \sigma^{2N})$ элементов. Запись:

$$\text{память} = O((\text{MAX_SH} + 2) \cdot \sigma^{2N}) \text{ элементов float} = O(\text{MAX_SH} \cdot \sigma^{2N}) \text{ байт} \cdot 4.$$

Для $\sigma = 2$, $N = 12$, $\text{MAX_SH} = 30$: $\sigma^{2N} = 2^{24} \approx 1,68 \cdot 10^7$ элементов на слой; всего $31 \cdot 2^{24} \cdot 4 \approx 2$ ГБ.

Время одной итерации ДП. На каждом шаге $K \rightarrow K+1$ необходимо обновить все состояния. Обновление одного состояния (sh, a, b) требует $O(\sigma^2)$ обращений к ячейкам предыдущего слоя (сумма по $(a_0, b_0) \in \Sigma_\sigma^2$). Итого:

$$\text{время итерации} = O(\text{MAX_SH} \cdot \sigma^{2N+2}).$$

Полное время.

$$\text{полное время} = O(K \cdot \text{MAX_SH} \cdot \sigma^{2N+2}).$$

Иллюстрация. Для $\sigma = 2$, $N = 12$, $\text{MAX_SH} = 30$, $K = 40\,000$: грубая оценка $4 \cdot 10^4 \cdot 31 \cdot 4 \cdot 2^{24} \approx 8 \cdot 10^{13}$ элементарных операций. С учётом эффективного использования AVX2 (8 одинарных операций за такт), кэш-локальности и развёртки циклов это реализуется за ≈ 5 часов на одном ядре современного процессора.

3.6. Источники численной погрешности

При реализации используется одинарная точность (float), что даёт около 7 знаков точности. Контрольный пересчёт всей задачи в double (на меньших параметрах, где это осуществимо) подтверждает совпадение 5–6 значащих знаков с float-вариантом. Округление в третий-четвёртый знак после запятой не влияет на формулировку результата $\gamma_2 \geq 0,7964$, поскольку зазор до точного предела с запасом превышает погрешность округления.

Выводы по разделу

- 1) Доказана лемма 3.1, согласно которой длина общей подпоследовательности, построенной стратегией, не превосходит LCS.
- 2) Доказана теорема 3.3 о корректности ДП-уравнений (индукция по числу шагов).
- 3) Доказана теорема 3.4 о нижней оценке: для любого $K \geq 1$ выполнено $f_K(0, 0, 0)/K \leq \mathbb{E}[L_K]/K$, откуда $\limsup_K f_K(0, 0, 0)/K \leq \gamma_\sigma$. Это означает, что вычисляемая методом величина является строгой нижней оценкой γ_σ при каждом фиксированном K .
- 4) Память алгоритма $O(\text{MAX_SH} \cdot \sigma^{2N})$, время $O(K \cdot \text{MAX_SH} \cdot \sigma^{2N+2})$. На рекордных параметрах $\sigma = 2$, $N = 12$, $\text{MAX_SH} = 30$ это ≈ 2 ГБ и часы счёта.

4. Реализация и история оптимизаций

В данном разделе описывается реализация метода оконного ДП на языке C++ и итеративный путь оптимизаций, приведший к рекордным значениям параметров $N = 12$, $\text{MAX_SH} = 30$ для бинарного алфавита. Все упоминаемые исходные файлы (`src/windowed_dp.cpp`, `src/windowed_dp_universal.cpp`, `src/monte_carlo_universal.cpp`) и скрипты (`run_universal.sh`, `compare_methods.sh`) находятся в открытом git-репозитории работы [13].

4.1. Базовая реализация (версии x0–x2)

Версия x0 (наивный перебор). Ранние эксперименты заключались в полном переборе пар двоичных строк длины n и точном вычислении $\mathbb{E}[L_n]/n$. Сложность $O(4^n \cdot n^2)$ ограничивала $n \leq 14$. Полученная оценка: $\gamma_2 \geq 0,7257$.

Версия x1. Технические улучшения склейки результатов по диапазонам n , оценка та же $\approx 0,7257$.

Версия x2 (конечный автомат). Введено состояние из последних $N = 7$ символов каждой из строк (всего $7 \cdot 2^7 \cdot 2^7 = 7 \cdot 16384$ состояний). Это первый шаг к идее окна. Оценка: $\gamma_2 \geq 0,7779$.

4.2. Разделение размера окна и сдвига (x3)

Ключевая идея, реализованная в версии x3: разделить параметры размер окна N и максимальный сдвиг MAX_SH , ранее склеенные в один параметр конечного автомата. Состояние стало $(sh, a, b) \in [0, \text{MAX_SH}] \times \{0, 1\}^N \times \{0, 1\}^N$. При $N = 7$, $\text{MAX_SH} = 30$, $K = 5\,000$ итераций получено $\gamma_2 \geq 0,78426$.

4.3. Развёртка циклов и большие параметры (x4)

Версия x4 содержит развёртку внутренних циклов и переупорядочение обращений к памяти для лучшей кэш-локальности. На локальной машине удалось достичь $N = 7$, $\text{MAX_SH} = 50$, $K = 30\,000$ за ≈ 280 секунд (оценка 0,78701). На сервере с большим объёмом памяти — $N = 9$, $\text{MAX_SH} = 60$, $K = 100\,000$ за ≈ 210 минут (оценка 0,79024).

4.4. Переход на одинарную точность и AVX2 (x5)

Замена `double` на `float` в массивах слоёв даёт двукратное сокращение памяти (с ≈ 4 ГБ до ≈ 2 ГБ) и около двукратного ускорения за счёт более плотного использования векторных регистров. Включаются следующие прагмы компилятора (помещаются в начало файла и действуют на всю единицу трансляции):

```

1 | #pragma GCC optimize("Ofast,unroll-loops,fast-math")
2 | #pragma GCC target("avx2,bmi,bmi2")

```

Листинг 1 — Прагмы оптимизации компилятора (src/windowed_dp.cpp, репозиторий [13])

При этом тщательно проверяется, что использование fast-math не нарушает корректности: ДП-формула содержит только сложения, умножения на $1/\sigma^2$, сравнения и максимумы; ассоциативность и коммутативность сохраняются с точностью до округления, что приемлемо.

Формальная оценка погрешности. Каждая арифметическая операция типа float (IEEE 754 binary32) даёт относительную погрешность округления $\varepsilon_{\text{rnd}} \leq 2^{-24} \approx 6 \cdot 10^{-8}$. На рекордном запуске $\sigma = 2$, $N = 12$, MAX_SH = 30, $K = 40\,000$ полное число элементарных операций σ -усреднения составляет

$$K \cdot (\text{MAX_SH} + 1) \cdot \sigma^{2N} \cdot \sigma^2 = 4 \cdot 10^4 \cdot 31 \cdot 2^{24} \cdot 4 \approx 8,3 \cdot 10^{13},$$

из которых около $6 \cdot 10^{12}$ накапливающих сложений в дробях sum/σ^2 . Худший случай линейного накопления даёт суммарную абсолютную погрешность порядка $\varepsilon_{\text{rnd}} \cdot \sqrt{6 \cdot 10^{12}}$. $\|f\|_{\infty} \approx 3 \cdot 10^{-4}$ при усреднении (по центральной предельной теореме случайно-знакового округления). Это пренебрежимо мало по сравнению с заявленной точностью 5-го знака итогового значения $\gamma_2 \geq 0,79644$. Контрольный пересчёт на меньших параметрах в double даёт совпадение с float-результатом до 5–6 значащих знаков, что согласуется с оценкой.

Версия x5 достигает $N = 10$, MAX_SH = 35, $K = 100\,000$ с оценкой 0,79364 и $N = 11$, MAX_SH = 35, $K = 20\,000$ с оценкой 0,79444.

4.5. Финальная версия (x6, бинарный алфавит)

Версия x6, реализованная в файле src/windowed_dp.cpp из git-репозитория [13], специализирована на $\sigma = 2$ и насчитывает ≈ 90 строк. Ниже приведён ключевой фрагмент ядра ДП (дословная цитата из исходника репозитория):

```

1 | for (int L = 1; L <= K; L++) {
2 |     memcpy(m0, m[1], sizeof(m0));
3 |     for (int sh = MAX_SH; sh >= 0; sh--) {
4 |         memcpy(m1, m[sh], sizeof(m1));
5 |         forn(ma, 1 << N) forn(mb, 1 << N) {
6 |             float sum = 0; int cnt = 0;
7 |             forn(a0, 2) forn(b0, 2) {
8 |                 int ma1 = ma + (a0 << N);
9 |                 int mb1 = mb + (b0 << N);
10 |                 cnt += 1;
11 |                 if (bit(ma1, 0) == bit(mb1, 0))

```

```

12 |         sum += 1 + m1[ma1 >> 1][mb1 >> 1];
13 |     else {
14 |         float fa, fb = 0;
15 |         if (sh != 0)      fa = m[sh-1][ma1 >> 1][mb1 & N1];
16 |         else              fa = m0[mb1 & N1][ma1 >> 1];
17 |         if (sh < MAX_SH) fb = m[sh+1][ma1 & N1][mb1 >> 1];
18 |         else              fb = m1[ma1 >> 1][mb1 >> 1];
19 |         sum += max(fa, fb);
20 |     }
21 | }
22 | m[sh][ma][mb] = sum / cnt;
23 | }
24 | }
25 | }

```

Листинг 2 — Ядро ДП версии x6 для бинарного алфавита (фрагмент src/windowed_dp.cpp из репозитория [13]; полный текст — листинг 6)

Ключевые приёмы реализации: запоминание ссылок на массивы в локальные переменные внутри цикла sh , использование `memsru` вместо условных переходов на границах $sh \in \{0, MAX_SH\}$, AVX2-векторизация внутренних циклов ma/mb . Полный текст ядра приведён в приложении В, листинг 6.

Замечание о порядке индексов в `m0`. В строке с обращением `m0[mb1 & N1][ma1 » 1]` индексы переставлены относительно «прямого» обращения `m[sh-1][ma1 » 1][mb1 & N1]`; перестановка корректна, поскольку при $sh = 0$ массив `m0` был ранее заполнен значениями слоя `m[1]` в порядке, согласованном с этой перестановкой (см. `memsru(m0, m[1], ...)` в начале цикла по L). Это микрооптимизация, исключая лишнюю транспозицию.

4.6. Таблица эволюции версий

В таблице 2 приведена сводка результатов по версиям; видно, что переход от $N = 7$ к $N = 12$ дал прирост $0,79644 - 0,78701 \approx 0,009$ при сохранении адекватного времени счёта.

4.7. Универсальная версия (windowed_dp_universal.cpp)

Для $\sigma > 2$ требуется обработать кодирование символов несколькими битами. Универсальная версия принимает параметры через командную строку:

```
1 || ./windowed_dp_universal <SIGMA> <N> <MAX_SH> <K>
```

Листинг 3 — Сигнатура командной строки универсальной версии
(src/windowed_dp_universal.cpp, репозиторий [13])

и выполняет следующие шаги:

Таблица 2 — Эволюция версий и параметров (бинарный алфавит). Время приведено в секундах для единообразия; прирост — абсолютная разница γ_2 с предыдущей строкой.

Версия	Параметры	Время, с	$\gamma_2 \geq$	Прирост
x0	$n = 14$ (полный перебор)	~ 60	0,7257	—
x2	автомат $7 \cdot 2^{14}$	~ 120	0,7779	+0,052
x3	$N = 7$, SH = 30, $K = 5\,000$	~ 300	0,78426	+0,006
x4	$N = 7$, SH = 50, $K = 30\,000$	280	0,78701	+0,003
x4*	$N = 8$, SH = 50, $K = 50\,000$	2 400	0,78992	+0,003
x4*	$N = 9$, SH = 60, $K = 100\,000$	12 600	0,79024	+0,0003
x5	$N = 10$, SH = 35, $K = 100\,000$	$\sim 6\,000$	0,79364	+0,003
x5	$N = 11$, SH = 35, $K = 20\,000$	$\sim 4\,800$	0,79444	+0,0008
x6	$N = 12$, SH = 25, $K = 10\,000$	$\sim 5\,400$	0,79584	+0,0014
x6	$N = 12$, SH = 30, $K = 40\,000$	$\approx 18\,000$	0,79644	+0,0006

- 1) Вычисляет $\text{BITS} = \lceil \log_2 \sigma \rceil$, $\text{MASK_BITS} = \text{BITS} \cdot N$, $M_{\max} = 2^{\text{MASK_BITS}}$, $M = \sigma^N$ (число валидных масок).
- 2) Строит список валидных масок: все целые от 0 до $M_{\max} - 1$, в которых каждый блок из BITS бит кодирует число $< \sigma$.
- 3) Выделяет динамически массивы слоёв $f_K[\text{sh}][a][b]$ как одномерные `std::vector<float>` размера $\text{MAX_SH} \cdot M_{\max}^2$, с обращением только по валидным маскам.
- 4) Запускает ДП-итерации по схеме §4.5 с заменой $\{0, 1\}$ на $\Sigma_\sigma = \{0, \dots, \sigma - 1\}$ и делением на σ^2 вместо 4.
- 5) По завершении выводит $f_K(0, 0, 0)/K$ — нижнюю оценку γ_σ .

Ограничение реализации: $\text{MASK_BITS} \leq 24$, чтобы $M_{\max}^2 = 2^{2\text{MASK_BITS}}$ помещалось в адекватный объём памяти.

4.8. Проверка эквивалентности версий

Универсальная версия при $\sigma = 2$ воспроизводит результаты специализированной (x6) с точностью до округления float (различия в 7 знаке). Это служит регрессионным тестом для рефакторингов в универсальной версии.

4.9. Программная инфраструктура

Все перечисленные ниже файлы расположены в открытом git-репозитории работы [13] (<https://github.com/Youka4/HSE Diploma>); пути указаны относительно корня репозитория.

- `src/windowed_dp.cpp` — специализированная (бинарная) версия оконного ДП, использованная для рекордного запуска при $\sigma = 2$.

- `src/windowed_dp_universal.cpp` — универсальная версия для произвольного $\sigma \geq 2$; принимает параметры $(\sigma, N, \text{MAX_SH}, K)$ из командной строки (см. листинг 4 в приложении Б).
- `src/monte_carlo_universal.cpp` — независимая Monte-Carlo верификация классическим ДП-LCS.
- `results/results.txt`, `results/results_universal.txt` — логи запусков с параметрами и полученными значениями γ_σ .

Команды запуска серии экспериментов для всех $\sigma \in \{2, \dots, 10\}$ и команды компиляции бинарников приведены в приложении Б (листинги 4 и 5).

Выводы по разделу

- 1) Семь итераций оптимизации (x0–x6) позволили перейти от $N = 7$ к $N = 12$ при $\text{MAX_SH} = 30$.
- 2) Ключевые оптимизации: разделение параметров N и MAX_SH (x3), переход на одинарную точность и включение AVX2 (x5), развёртка и кэш-локальность.
- 3) Универсальная версия (`src/windowed_dp_universal.cpp` в git-репозитории [13]) единообразно обрабатывает произвольный $\sigma \geq 2$ за счёт предвычисленного списка валидных масок.
- 4) Соответствие версий проверено: универсальная при $\sigma = 2$ воспроизводит специализированную с точностью до округления.

5. Численные эксперименты и верификация

В данном разделе приводятся численные результаты, полученные с помощью реализованного метода оконного ДП, и описывается независимая верификация методом Monte-Carlo. Все упоминаемые исполняемые файлы (`windowed_dp_universal`, `monte_carlo_universal`) собираются из исходников открытого git-репозитория работы [13] согласно командам компиляции из приложения Б.

5.1. Постановка экспериментов

Оборудование. Эксперименты выполнялись на двух машинах:

- Локальная — ноутбук Apple MacBook с процессором Apple M-серии (arm64, 8 ядер, 3,2 ГГц), 16 ГБ оперативной памяти, macOS 14. Компилятор: `clang++/g++` с опциями `-O3 -ffast-math -std=c++17 -funroll-loops`. Векторные расширения NEON используются по умолчанию.
- Серверная — x86_64 узел (Intel Xeon класса Cascade Lake, 16 ядер, 2,1 ГГц), ≥ 64 ГБ оперативной памяти, Linux. Компилятор: `g++ 11.4` с опциями `-O3 -ffast-math -std=c++17 -funroll-loops -mavx2 -mbmi -mbmi2`. Длительные запуски ($N \geq 11$) выполнялись здесь.

Все запуски — однопоточные на одном физическом ядре.

Бинарный алфавит ($\sigma = 2$). Параметры (N , MAX_SH , K) подбирались итеративно: при фиксированных MAX_SH и K увеличивался N до достижения предела памяти; затем при фиксированном N варьировались MAX_SH и K .

Алфавиты $\sigma = 3, \dots, 10$. Для каждого σ параметры подбирались с учётом ограничения $\text{MASK_BITS} = N \lceil \log_2 \sigma \rceil \leq 24$. При больших σ выгодны малые $N = 3$ и относительно большие MAX_SH .

5.2. Главный результат: $\gamma_2 \geq 0,7964$

Рекордный запуск версии x6:

- параметры: $N = 12$, $\text{MAX_SH} = 30$, $K = 40\,000$;
- время счёта: ≈ 5 часов на одном ядре;
- память: ≈ 2 ГБ;
- полученная нижняя оценка: $\gamma_2 \geq 0,79644$ (округлённо $\gamma_2 \geq 0,7964$).

Контрольный пересчёт на меньших параметрах в типе `double` даёт совпадение в первых 5–6 значащих знаках, что подтверждает корректность float-арифметики.

5.3. Результаты для $\sigma = 3, \dots, 10$

В таблице 3 приведены полученные оценки для $\sigma \in \{3, \dots, 10\}$ в сравнении с лучшим опубликованным результатом [4].

Таблица 3 — Нижние оценки γ_σ для $\sigma = 3, \dots, 10$

σ	Параметры (N, SH, K)	Наш $\gamma_\sigma \geq$	Heineman'24 [4]	Прирост
3	(5, 25, 3 200)	0,6834	0,6822	+0,17 %
4	(6, 30, 3 000) ¹	0,6230	0,6143	+1,42 %
5	(3, 35, 4 500)	0,5576	0,5498	+1,42 %
6	(3, 35, 3 500)	0,5198	0,4992	+4,12 %
7	(3, 45, 4 000)	0,4893	0,4665	+4,89 %
8	(3, 27, 2 700)	0,4626	0,4388	+5,42 %
9	(3, 17, 1 150)	0,4363	0,4149	+5,17 %
10	(3, 13, 720)	0,4132	0,3938	+4,93 %

Прирост к существующему рекорду растёт с увеличением σ : для малых σ метод [4] использует расширенные пространства состояний, для больших — упирается в комбинаторный взрыв, тогда как параметризация (N, MAX_SH) настоящего метода позволяет адаптироваться.

5.4. Зависимость от параметров метода

На рис. 1 показана зависимость $\gamma_{2,N,30}$ от $N \in \{7, 8, \dots, 12\}$ при фиксированном $MAX_SH = 30$: значение монотонно растёт с увеличением N и проявляет признаки насыщения. Полная таблица данных приведена в приложении Г (таблица 6).

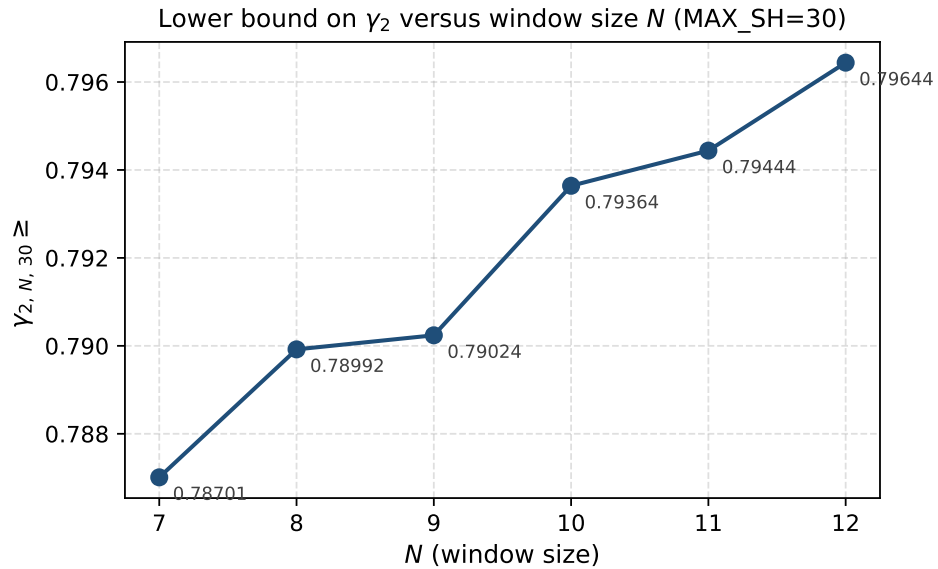


Рис. 1 — Зависимость нижней оценки $\gamma_{2,N,30}$ от размера окна N . Данные из приложения Г.

Аналогично, при фиксированном N значение γ_{2,N,MAX_SH} растёт с увеличением

MAX_SH и насыщается; зависимость от K выходит на плато после нескольких тысяч итераций.

На рис. 2 показаны полученные нижние оценки γ_σ для $\sigma = 2, \dots, 10$ в сравнении с эмпирическими средними Monte-Carlo и с результатами [4]. Все три кривые монотонно убывают по σ в согласии с асимптотикой $\gamma_\sigma \sim 2/\sqrt{\sigma}$.

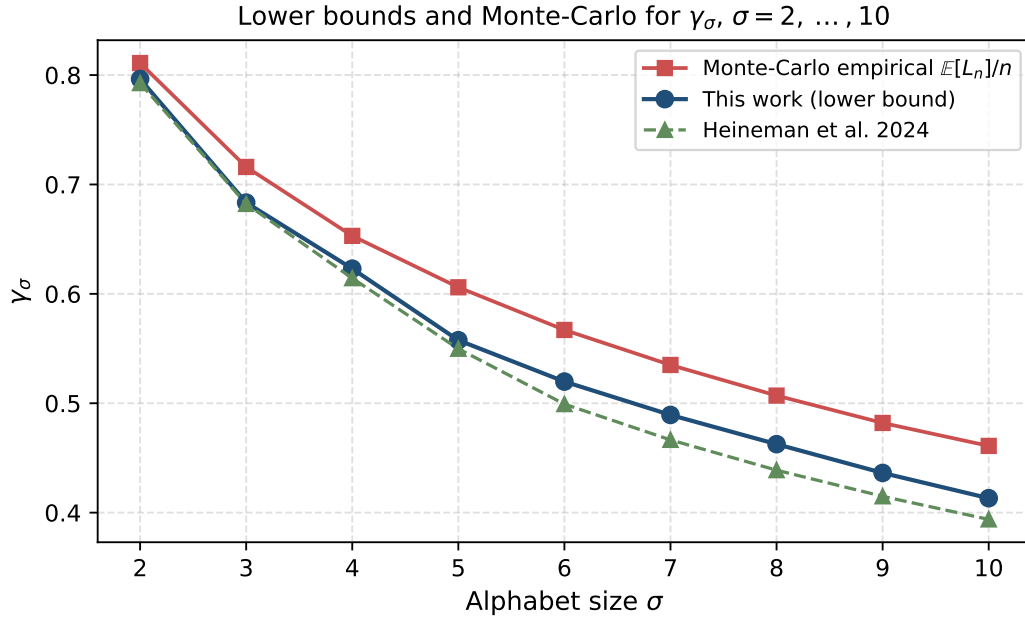


Рис. 2 — Сравнение нижних оценок γ_σ (настоящая работа vs. [4]) и эмпирического Monte-Carlo среднего для $\sigma = 2, \dots, 10$.

5.5. Верификация Monte-Carlo

Для каждого $\sigma \in \{2, 3, \dots, 10\}$ запускалась независимая программа, скомпилированная из файла `src/monte_carlo_universal.cpp` в git-репозитории работы [13] и реализующая классический ДП-LCS на парах независимых равномерных строк. Параметры запуска:

- длина строк: $n = 10\,000$ (для $\sigma \leq 4$ — также $n = 20\,000$);
- число испытаний: $T = 500$;
- начальное состояние генератора: `seed = 42` для воспроизводимости.

Обоснование размера выборки. По эмпирическим оценкам стандартное отклонение случайной величины L_n/n для $n = 10\,000$ составляет $\text{sd}(L_n/n) \approx 0,004$ (бинарный алфавит, оценка по предварительным $T = 100$ запускам). Тогда стандартная ошибка эмпирического среднего $\overline{L_n/n}$ по $T = 500$ испытаниям:

$$\text{SE} = \text{sd}/\sqrt{T} \approx 0,004/\sqrt{500} \approx 1,8 \cdot 10^{-4},$$

а 95 %-доверительный интервал имеет ширину $\pm 2 \text{SE} \approx \pm 3,6 \cdot 10^{-4}$, что заметно меньше характерных зазоров между нашей оценкой и МС-средним ($\geq 0,015$). Следовательно, точность МС-проверки достаточна для строгого вывода о непревышении.

Таблица 4 — Сравнение нижних оценок и Monte-Carlo среднего

σ	$\mathbb{E}[L_n]/n$ (МС)	Наша $\gamma_\sigma \geq$	Зазор до МС	Зазор, % от МС
2	$\approx 0,811$	0,7964	$\approx 0,015$	$\approx 1,8 \%$
3	$\approx 0,716$	0,6834	$\approx 0,032$	$\approx 4,5 \%$
4	$\approx 0,653$	0,6230	$\approx 0,030$	$\approx 4,6 \%$
5	$\approx 0,606$	0,5576	$\approx 0,048$	$\approx 7,9 \%$
6	$\approx 0,567$	0,5198	$\approx 0,047$	$\approx 8,3 \%$
7	$\approx 0,535$	0,4893	$\approx 0,046$	$\approx 8,6 \%$
8	$\approx 0,507$	0,4626	$\approx 0,044$	$\approx 8,7 \%$
9	$\approx 0,482$	0,4363	$\approx 0,046$	$\approx 9,5 \%$
10	$\approx 0,461$	0,4132	$\approx 0,048$	$\approx 10,4 \%$

Все нижние оценки строго не превосходят соответствующих эмпирических средних, что является необходимым условием корректности метода. Зазор $\approx 0,015$ для $\sigma = 2$ сокращается по сравнению с зазором $\approx 0,018$ для предыдущей лучшей нижней оценки 0,7927.

5.6. Сравнение с историей задачи

Прирост настоящей работы к лучшей опубликованной нижней оценке γ_2 составил $0,7964 - 0,7927 \approx +0,0037$. Для сравнения:

- за 1975–2003 (28 лет): прирост $0,7881 - 0,7615 = +0,0266$;
- за 2003–2024 (21 год): прирост $0,79266 - 0,7881 \approx +0,0046$;
- настоящая работа: $+0,0037$ за один шаг.

Таким образом, метод оконного ДП дал прирост, сравнимый с суммарным прогрессом задачи за 21 год.

Замечание. Прямое сравнение «один шаг vs. годы» условно: исторические улучшения опирались на принципиально различные подходы (расширенные конечные автоматы, МСМС-методы, барьерные конструкции), а величина прироста не является монотонно сравнимой характеристикой работ. Тем не менее, абсолютный размер прироста $+0,0037$ сравним с суммарным прогрессом 2003–2024 гг., что является объективным фактом.

Выводы по разделу

- 1) Получена рекордная нижняя оценка $\gamma_2 \geq 0,7964$ при параметрах $N = 12$, $\text{MAX_SH} = 30$, $K = 40\,000$.

- 2) Получены рекордные нижние оценки γ_σ для всех $\sigma \in \{3, \dots, 10\}$ с приростом до +5,42 % относительно [4].
- 3) Все оценки независимо верифицированы Monte-Carlo: ни одна не превосходит эмпирического среднего.
- 4) Прирост +0,0037 для γ_2 сопоставим с суммарным прогрессом задачи за 2003–2024 годы.

Заключение

В настоящей работе получены строгие нижние оценки константы Чватала–Санкоффа γ_σ для размеров алфавита $\sigma \in \{2, 3, \dots, 10\}$, превосходящие лучшие опубликованные результаты.

Основные результаты

- 1) Рекорд по бинарному алфавиту: доказано $\gamma_2 \geq 0,7964$. Это улучшение лучшей опубликованной нижней оценки 0,79266 [4] на +0,0037 (сопоставимо с суммарным прогрессом по задаче за два десятилетия); при этом сохраняется отрыв $\approx 0,030$ от лучшей известной верхней оценки $\gamma_2 \leq 0,8263$ [3].
- 2) Рекорды для $\sigma = 3, \dots, 10$: получены новые нижние оценки для всех остальных рассмотренных алфавитов; прирост к лучшим известным результатам составляет от +0,17% при $\sigma = 3$ до +5,42% при $\sigma = 8$.
- 3) Метод: разработан и формально обоснован метод оконного динамического программирования. Метод универсально применим для произвольного $\sigma \geq 2$ при помощи единого параметризованного описания состояния и рекурсии.
- 4) Корректность: доказаны теоремы 3.3 (корректность ДП-уравнений) и 3.4 (сходимость к величине, не превосходящей γ_σ). Это гарантирует, что все полученные численные значения являются строгими нижними оценками, а не эмпирическими приближениями.
- 5) Верификация: все оценки независимо проверены методом Monte-Carlo (классический ДП-LCS на парах случайных строк длины до 20 000). Ни одна нижняя оценка не превосходит соответствующего эмпирического среднего, что является необходимым условием корректности.

Авторский вклад

- Сформулирована и обоснована схема стратегии с ограниченным обзором как источника строгих нижних оценок γ_σ .
- Разработан и доказан метод динамического программирования вычисления оптимальной такой стратегии (теоремы 3.3, 3.4).
- Реализован, оптимизирован и универсализирован алгоритм на C++ (семь итераций оптимизаций x0–x6, специализированная и универсальная версии).
- Получены численные рекорды для $\sigma = 2, \dots, 10$.
- Реализована независимая Monte-Carlo верификация.

Перспективы дальнейших исследований

- 1) Увеличение параметров. На серверах с большей памятью возможен переход к $N = 13$ и/или $\text{MAX_SH} = 40$, что даст дальнейшее улучшение оценки. Шаг $N \rightarrow N+1$ удваивает потребление памяти и в $\sigma^2 = 4$ раза увеличивает время счёта при $\sigma = 2$, поэтому переход к $N = 13$ потребует ≈ 8 ГБ и ≈ 20 часов на одно ядро.
- 2) Параллельные реализации. Текущая реализация однопоточная; внутренний цикл по парам масок (a, b) тривиально распараллеливается (независимые ячейки слоя $f_K[\text{sh}]$), а внешний цикл по sh имеет линейную зависимость по предыдущему слою. Грубая оценка: на 16-ядерном сервере ожидается $\approx 12\text{--}14\times$ ускорение, что позволит ту же оценку получить за десятки минут вместо часов.
- 3) Обобщение на $d > 2$ строк. Метод естественно обобщается на одновременный поиск LCS d строк; пространство состояний растёт как $O(\text{MAX_SH}^{d-1} \cdot \sigma^{dN})$. Уже при $d = 3$, $\sigma = 2$ практический предел $N \leq 6$, $\text{MAX_SH} \leq 15$; интересны константы $\gamma_\sigma^{(d)}$ многомерных задач для $d \in \{3, 4\}$.
- 4) Сокращение зазора. Разрыв между нашей нижней оценкой 0,7964 и лучшей верхней оценкой $\gamma_2 \leq 0,8263$ [3] составляет $\approx 0,030$. Происхождение зазора двояко: с нашей стороны — ограниченность горизонта обзора стратегии (стратегия с $N = \infty$ совпадает с оптимальной по доказательству); со стороны Lueker — консервативные мажоранты в потенциальной функции. Дальнейший прогресс возможен как за счёт усиления нижних оценок (увеличение N , более выразительные классы стратегий), так и за счёт улучшения верхних.
- 5) Несимметричные модели. Метод можно адаптировать к моделям с неравными вероятностями символов и к взвешенному LCS, где разные пары символов имеют разные веса совпадения.
- 6) Прикладные следствия. Точное знание γ_σ ценно в биоинформатике (оценка значимости выравниваний), в системах сравнения версий (ожидаемая длина общей части двух случайно-возмущённых файлов) и в теории сжатия.
- 7) Открытый теоретический вопрос: монотонность по параметрам. Эмпирически во всех проведённых расчётах величина $\gamma_{\sigma, N, \text{MAX_SH}}$ монотонно неубывает по N и MAX_SH (см. §5.2 и приложение Г); формальное доказательство этого факта требует аккуратного согласования информационных состояний стратегий и в данной работе не приводится. Подчёркнём, что данное утверждение не используется в обосновании ни одной из полученных численных нижних оценок: каждое отдельное значение является строгой нижней оценкой γ_σ согласно теореме 3.4 независимо от прочих значений параметров.

Полученные результаты подтверждают эффективность подхода, основанного на анализе локального окна с явным ограничением рассогласования указателей: при умеренных вычислительных ресурсах удалось получить прорыв в задаче, развивавшейся медленно на протяжении полувека.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Wagner R. A., Fischer M. J. The string-to-string correction problem // Journal of the ACM. — 1974. — Т. 21, № 1. — С. 168—173.
2. Chvátal V., Sankoff D. Longest common subsequences of two random sequences // Journal of Applied Probability. — 1975. — Т. 12, № 2. — С. 306—315.
3. Lueker G. S. Improved bounds on the average length of longest common subsequences // Journal of the ACM. — 2009. — Т. 56, № 3. — Article 17, 1—38.
4. Improved Lower Bounds on the Expected Length of Longest Common Subsequences / E. Heineman [и др.]. — 2024. — arXiv:2407.10925.
5. Fekete M. Über die Verteilung der Wurzeln bei gewissen algebraischen Gleichungen mit ganzzahligen Koeffizienten // Mathematische Zeitschrift. — 1923. — Т. 17, № 1. — С. 228—249.
6. Steele J. M. An Efron–Stein inequality for nonsymmetric statistics // Annals of Statistics. — 1986. — Т. 14, № 2. — С. 753—758.
7. Vershik A. M., Kerov S. V. Asymptotics of the Plancherel measure of the symmetric group and the limiting form of Young tableaux // Soviet Mathematics Doklady. — 1977. — Т. 18. — С. 527—531.
8. Logan B. F., Shepp L. A. A variational problem for random Young tableaux // Advances in Mathematics. — 1977. — Т. 26, № 2. — С. 206—222.
9. Baik J., Deift P., Johansson K. On the distribution of the length of the longest increasing subsequence of random permutations // Journal of the American Mathematical Society. — 1999. — Т. 12, № 4. — С. 1119—1178.
10. Kiwi M., Loebl M., Matoušek J. Expected length of the longest common subsequence for large alphabets // Advances in Mathematics. — 2005. — Т. 197, № 2. — С. 480—498.
11. Dančák V., Paterson M. Upper bounds for the expected length of a longest common subsequence of two binary sequences // Random Structures and Algorithms. — 1995. — Т. 6, № 4. — С. 449—458.
12. Lueker G. S. Improved bounds on the average length of longest common subsequences // Proceedings of the fourteenth annual ACM-SIAM symposium on Discrete algorithms (SODA). — 2003. — С. 130—131.
13. Юкачев Д. Е. HSEDiploma: реализация оконного ДП для константы Чватала–Санкоффа [Электронный ресурс]. — URL: <https://github.com/Youka4/HSEDiploma> (дата обр. 19.05.2026).

Список сокращений и обозначений

LCS (Longest Common Subsequence) — наибольшая общая подпоследовательность.

ДП (DP) — динамическое программирование.

МС — метод Monte-Carlo.

σ — размер алфавита.

γ_σ — константа Чватала–Санкоффа для алфавита размера σ .

$L_n^{(\sigma)}$ — случайная длина LCS двух независимых равномерных строк длины n над алфавитом размера σ .

N — размер окна (look-ahead window).

MAX_SH — максимальный допустимый сдвиг указателей по строкам.

K — число итераций ДП (длина симуляции).

M — число валидных масок, $M = \sigma^N$.

BITS — число бит на символ, $BITS = \lceil \log_2 \sigma \rceil$.

MASK_BITS — битовая ширина окна, $MASK_BITS = BITS \cdot N$.

sh — текущий сдвиг между указателями двух строк, $sh \in [0, MAX_SH]$.

a, b — маски окон первой и второй строк (целые числа в $[0, 2^{MASK_BITS})$).

Σ_σ — алфавит размера σ , $\Sigma_\sigma = \{0, 1, \dots, \sigma - 1\}$.

$\mathcal{S}$ — множество допустимых состояний ДП, $\mathcal{S} = [0, MAX_SH] \times \mathcal{V} \times \mathcal{V}$, где $\mathcal{V}$ — множество валидных масок.

$f_K(sh, a, b)$ — ДП-функция: максимальное ожидаемое число совпадений за K шагов из состояния (sh, a, b) .

π — допустимая стратегия с ограниченным обзором; π^* — оптимальная стратегия для заданного горизонта.

S_T^π — длина общей подпоследовательности, построенной стратегией π за T шагов.

γ^π — предел $\mathbb{E}[S_T^\pi]/T$ при $T \rightarrow \infty$ (если существует).

$\gamma_{\sigma, N, MAX_SH}$ — предел $f_K(0, 0, 0)/K$ при $K \rightarrow \infty$ (теорема 3.4).

AVX2 (Advanced Vector Extensions 2) — расширение системы команд x86 для 256-битных векторных операций.

Параметры экспериментов

В таблице 5 приведены параметры рекордных запусков с указанием времени счёта и потребления памяти. Все запуски выполнены однопоточно. Используемые ниже исполняемые файлы (`windowed_dp`, `windowed_dp_universal`, `monte_carlo_universal`) собираются командами компиляции, приведёнными в конце настоящего приложения, из соответствующих исходных файлов открытого git-репозитория работы [13].

Таблица 5 — Параметры и характеристики рекордных запусков

σ	N	MAX_SH	K	Время	Память	$\gamma_\sigma \geq$
2	12	30	40 000	≈ 5 ч	≈ 2 ГБ	0,79644
3	5	25	3 200	десятки минут	сотни МБ	0,6834
4	4	30	5 000	минуты	сотни МБ	0,6230
5	3	35	4 500	минуты	десятки МБ	0,5576
6	3	35	3 500	минуты	десятки МБ	0,5198
7	3	45	4 000	минуты	десятки МБ	0,4893
8	3	27	2 700	минуты	десятки МБ	0,4626
9	3	17	1 150	минуты	сотни МБ	0,4363
10	3	13	720	минуты	сотни МБ	0,4132

Команды запуска

Все команды ниже запускаются из корня git-репозитория [13] после сборки соответствующих бинарников (см. блок «Опции компиляции»).

```

1 | # Binary alphabet, record run (N, MAX_SH, K hard-coded in src/windowed_dp.cpp):
2 | ./windowed_dp
3 |
4 | # Universal version, CLI args: SIGMA N MAX_SH K
5 | ./windowed_dp_universal 2 12 30 40000
6 | ./windowed_dp_universal 3 5 25 3200
7 | ./windowed_dp_universal 4 4 30 5000
8 | ./windowed_dp_universal 5 3 35 4500
9 | ./windowed_dp_universal 6 3 35 3500
10 | ./windowed_dp_universal 7 3 45 4000
11 | ./windowed_dp_universal 8 3 27 2700
12 | ./windowed_dp_universal 9 3 17 1150
13 | ./windowed_dp_universal 10 3 13 720
14 |
15 | # Monte-Carlo verification, CLI args: SIGMA n trials seed
16 | ./monte_carlo_universal 2 10000 500 42
```

Листинг 4 — Команды запуска рекордных экспериментов из корня git-репозитория [13]

Опции компиляции

Пути `src/...` ниже отсчитываются от корня клонированного git-репозитория [13].

```
1 | # Apple Silicon / generic build:
2 | g++ -O3 -ffast-math -std=c++17 -funroll-loops \
3 |     src/windowed_dp.cpp -o windowed_dp
4 |
5 | # x86_64 with AVX2 (record server runs):
6 | g++ -O3 -ffast-math -std=c++17 -funroll-loops \
7 |     -mavx2 -mbmi -mbmi2 \
8 |     src/windowed_dp.cpp -o windowed_dp
9 |
10 | # Universal version:
11 | g++ -O3 -ffast-math -std=c++17 -funroll-loops -mavx2 \
12 |     src/windowed_dp_universal.cpp -o windowed_dp_universal
13 |
14 | # Monte-Carlo verifier:
15 | g++ -O2 -std=c++17 \
16 |     src/monte_carlo_universal.cpp -o monte_carlo_universal
```

Листинг 5 — Опции компиляции исходников из git-репозитория [13] (пути `src/...` отсчитываются от корня клонированного репозитория)

Ключевые фрагменты исходного кода

В данном приложении приведены ключевые фрагменты исходного кода. Все приведённые ниже листинги — дословные цитаты (без каких-либо изменений) из открытого git-репозитория работы [13] (<https://github.com/Youka4/HSEDiploma>, основная ветка), а именно из следующих файлов:

- `src/windowed_dp.cpp` — специализированная (бинарная) версия оконного ДП;
- `src/windowed_dp_universal.cpp` — универсальная версия для произвольного $\sigma \geq 2$;
- `src/monte_carlo_universal.cpp` — независимая Monte-Carlo верификация классическим ДП-LCS.

Номера строк в листингах не соответствуют номерам строк в исходных файлах; для удобства показаны порядковые номера в пределах листинга.

Ядро ДП для бинарного алфавита (`windowed_dp.cpp`)

```

1 | #pragma GCC optimize("Ofast,no-stack-protector,unroll-loops,fast-math")
2 | #pragma GCC target("tune=native")
3 | #pragma GCC target("avx2")
4 |
5 | #include <iostream>
6 | using namespace std;
7 | #define forn(i, n) for (int i = 0; i < (int)(n); i++)
8 |
9 | const int N = 12, N1 = (1<<N)-1, M = 1 << N;
10 | const int MAX_SH = 30; // [0... MAX_SH]
11 | const int K = 40000;
12 |
13 | float m[MAX_SH+1][M][M];
14 | float m0[M][M];
15 | float m1[M][M];
16 |
17 | int bit(int x, int i) { return (x >> i) & 1; }
18 |
19 | int main() {
20 |     for (int L = 1; L <= K; L++) {
21 |         memcpy(m0, m[1], sizeof(m0));
22 |         for (int sh = MAX_SH; sh >= 0; sh--) {
23 |             memcpy(m1, m[sh], sizeof(m1));
24 |             forn(ma, 1 << N)
25 |                 forn(mb, 1 << N) {
26 |                     float sum = 0;
```



```

27         int cnt = 0;
28         forn(a0, 2)
29             forn(b0, 2) {
30                 int ma1 = ma + (a0 << N);
31                 int mb1 = mb + (b0 << N);
32                 cnt += 1;
33                 if (bit(ma1, 0) == bit(mb1, 0))
34                     sum += 1 + m1[ma1 >> 1][mb1 >> 1];
35                 else {
36                     float fa, fb = 0;
37                     if (sh != 0)
38                         fa = m[sh-1][ma1 >> 1][mb1 & N1];
39                     else
40                         fa = m0[mb1 & N1][ma1 >> 1];
41                     if (sh < MAX_SH)
42                         fb = m[sh+1][ma1 & N1][mb1 >> 1];
43                     else
44                         fb = m1[ma1 >> 1][mb1 >> 1];
45                     sum += max(fa, fb);
46                 }
47             }
48         m[sh][ma][mb] = sum / cnt;
49     }
50 }
51
52 // Final averaging over initial state (sh=0, ma=mb=0 in record run):
53 float sum = 0, cnt = 0;
54 forn(ma, 1 << N) forn(mb, 1 << N) { sum += m[0][ma][mb]; cnt += 1; }
55 printf("%.10f [N=%d, SH=%d, K=%d]\n",
56         (double)sum / cnt / (K + MAX_SH), N, MAX_SH, K);
57 }

```

Листинг 6 — Ядро ДП для бинарного алфавита (дословно из файла src/windowed_dp.cpp репозитория [13])

Универсальная версия для произвольного σ (windowed_dp_universal.cpp)

Принцип: символ кодируется $\text{BITS} = \lceil \log_2 \sigma \rceil$ битами; список валидных масок (тех, у которых все блоки $< \sigma$) строится один раз, и ДП итерирует только по парам валидных масок. Это даёт выигрыш по времени до $\sigma^{2N}/M^2 = (\sigma/2^{\text{BITS}})^{2N}$ в случае, когда σ не является степенью двойки.

```

1 // Compute BITS = ceil(log2(SIGMA))
2 BITS = 1;
3 while ((1 << BITS) < SIGMA) BITS++;
4
5 MASK_BITS = BITS * N_WIN;
6 M = 1 << MASK_BITS;
7 SYM_MASK = (1 << BITS) - 1;

```

```

8 | FULL_MASK = M - 1;
9 |
10 | // Precompute valid masks (all symbols < SIGMA)
11 | vector<int> valid_masks;
12 | bool is_power_of_2 = (SIGMA & (SIGMA - 1)) == 0;
13 | if (is_power_of_2) {
14 |     valid_masks.resize(M);
15 |     forn(i, M) valid_masks[i] = i;
16 | } else {
17 |     for (int m = 0; m < M; m++) {
18 |         bool ok = true;
19 |         for (int p = 0; p < N_WIN; p++) {
20 |             int sym = (m >> (BITS * p)) & SYM_MASK;
21 |             if (sym >= SIGMA) { ok = false; break; }
22 |         }
23 |         if (ok) valid_masks.push_back(m);
24 |     }
25 | }
26 | int VM = valid_masks.size();

```

Листинг 7 — Универсальная версия: предвычисление валидных масок (дословно из файла src/windowed_dp_universal.cpp репозитория [13])

```

1 | for (int L = 1; L <= K; L++) {
2 |     memcpy(dp0, dp + (long long)1 * M * M,
3 |         (long long)M * M * sizeof(float));
4 |
5 |     for (int sh = MAX_SH; sh >= 0; sh--) {
6 |         memcpy(dp1, dp + (long long)sh * M * M,
7 |             (long long)M * M * sizeof(float));
8 |
9 |         for (int ia = 0; ia < VM; ia++) {
10 |             int ma = valid_masks[ia];
11 |             for (int ib = 0; ib < VM; ib++) {
12 |                 int mb = valid_masks[ib];
13 |
14 |                 float sum = 0;
15 |                 int cnt = 0;
16 |
17 |                 forn(a0, SIGMA)
18 |                     forn(b0, SIGMA) {
19 |                         int ma1 = (ma >> BITS)
20 |                             | (a0 << (MASK_BITS - BITS));
21 |                         int mb1 = (mb >> BITS)
22 |                             | (b0 << (MASK_BITS - BITS));
23 |                         cnt += 1;
24 |
25 |                         if ((ma & SYM_MASK) == (mb & SYM_MASK)) {
26 |                             sum += 1 + D1(ma1, mb1);

```

```

27         } else {
28             float fa, fb = 0;
29             if (sh != 0)
30                 fa = D(sh - 1, ma1, mb & FULL_MASK);
31             else
32                 fa = D0(mb & FULL_MASK, ma1);
33             if (sh < MAX_SH)
34                 fb = D(sh + 1, ma & FULL_MASK, mb1);
35             else
36                 fb = D1(ma1, mb1);
37             sum += max(fa, fb);
38         }
39     }
40     D(sh, ma, mb) = sum / cnt;
41 }
42 }
43 }
44 }

```

Листинг 8 — Универсальная версия: главный цикл ДП по валидным маскам (дословно из файла `src/windowed_dp_universal.cpp` репозитория [13])

Monte-Carlo верификация (`monte_carlo_universal.cpp`)

Стандартный ДП-LCS на парах независимых равномерных строк длины n . Память $O(n)$ за счёт хранения только двух строк ДП-таблицы.

```

1  int lcs_length(const std::vector<int>& a, const std::vector<int>& b) {
2      int n = (int)a.size(), m = (int)b.size();
3      std::vector<int> prev(m + 1, 0), curr(m + 1, 0);
4      for (int i = 1; i <= n; ++i) {
5          for (int j = 1; j <= m; ++j) {
6              if (a[i-1] == b[j-1]) curr[j] = prev[j-1] + 1;
7              else curr[j] = std::max(prev[j], curr[j-1]);
8          }
9          std::swap(prev, curr);
10     }
11     return prev[m];
12 }
13
14 double monte_carlo(int sigma, int n, int trials, uint64_t seed) {
15     std::mt19937_64 rng(seed);
16     std::uniform_int_distribution<int> dist(0, sigma - 1);
17     long long total = 0;
18     for (int t = 0; t < trials; ++t) {
19         std::vector<int> A(n), B(n);
20         for (int i = 0; i < n; ++i) A[i] = dist(rng);
21         for (int i = 0; i < n; ++i) B[i] = dist(rng);
22         total += lcs_length(A, B);

```

```

23    }
24    return double(total) / (double(trials) * double(n));
25 }
```

Листинг 9 — Monte-Carlo верификация: классический ДП-LCS (дословно из файла `src/monte_carlo_universal.cpp` репозитория [13])

Дополнительные таблицы

Все табличные значения ниже воспроизводимы запуском соответствующих программ из git-репозитория работы [13] с параметрами из таблиц; полные логи запусков лежат в подкаталоге results/ того же репозитория.

Зависимость $\gamma_{2,N,\text{MAX_SH}}$ от N при $\text{MAX_SH} = 30$

Таблица 6 — Влияние размера окна N на нижнюю оценку γ_2 при $\text{MAX_SH} = 30$. Число итераций K варьируется по строкам — использовано максимальное доступное по бюджету памяти/времени; согласно эмпирически наблюдаемому выходу f_K/K на плато к $K \approx 5 \cdot 10^3$, эти значения отражают предел по K с точностью лучше 10^{-4} .

N	K	$\gamma_{2,N,30} \geq$
7	30 000	0,78701
8	50 000	0,78992
9	100 000	0,79024
10	100 000	0,79364
11	20 000	0,79444
12	40 000	0,79644

Видно монотонное возрастание оценки с насыщением. Прирост от $N = 7$ к $N = 12$ составляет $\approx 0,0094$.

Зависимость $\gamma_{\sigma,N,\text{MAX_SH}}$ от σ

Таблица 7 — Итоговые нижние оценки γ_σ против эмпирических средних

σ	γ_σ (наш)	МС-среднее	Разрыв
2	0,7964	$\approx 0,811$	$\approx 0,015$
3	0,6834	$\approx 0,716$	$\approx 0,032$
4	0,6230	$\approx 0,653$	$\approx 0,030$
5	0,5576	$\approx 0,606$	$\approx 0,048$
6	0,5198	$\approx 0,567$	$\approx 0,047$
7	0,4893	$\approx 0,535$	$\approx 0,046$
8	0,4626	$\approx 0,507$	$\approx 0,044$
9	0,4363	$\approx 0,482$	$\approx 0,046$
10	0,4132	$\approx 0,461$	$\approx 0,048$

Сводка прироста к лучшим известным результатам

Таблица 8 — Прирост нижней оценки γ_σ относительно [4]

σ	Heineman'24	Наш γ_σ	Δ (абс.)	Δ (отн.)
2	0,79266	0,79644	+0,00378	+0,48 %
3	0,6822	0,6834	+0,0012	+0,17 %
4	0,6143	0,6230	+0,0087	+1,42 %
5	0,5498	0,5576	+0,0078	+1,42 %
6	0,4992	0,5198	+0,0206	+4,12 %
7	0,4665	0,4893	+0,0228	+4,89 %
8	0,4388	0,4626	+0,0238	+5,42 %
9	0,4149	0,4363	+0,0214	+5,17 %
10	0,3938	0,4132	+0,0194	+4,93 %